Regularization

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State

Professor, Electrical and Computer (ECE) Engineering, NC State

NAIRR AI Education Fellow, Computing Research Association (CRA)

Probabilistic Intuition

- One way to define a loss is to consider the **log likelihood**.

$$l(\theta|y,x) = \log(P_\theta(\hat{y} = y|x,\theta))$$

- Remember that for classification, the model returns the probability for a class, so in that case, the loss can be specified to be

$$L(y,\hat{y}) = -l(\theta|y,x)$$

- By making a few assumptions about distributions, this leads to the use of MSE for regression (assuming some Gaussian Distributions), and the use of cross-entropy for classification.

Probabilistic Intuition

- However, this does account for any **priors** on θ. If we take that into account, then we would end up with an expression of the loss as follows:

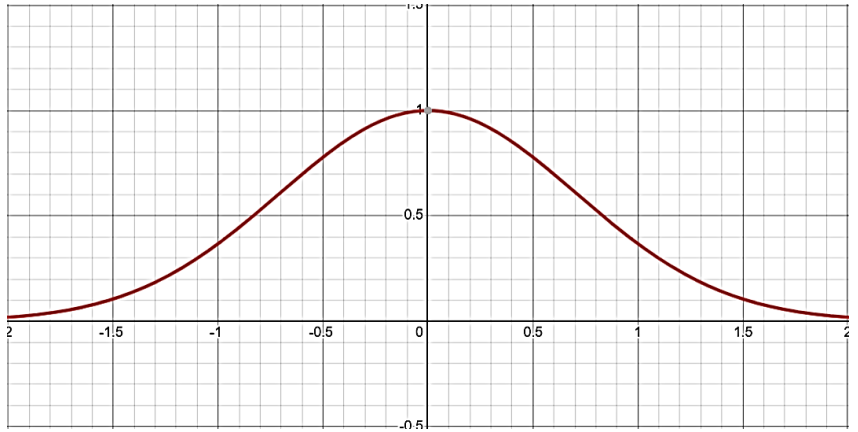
$$L(y, \hat{y}) = -l(\theta|y, x) - \log\big(P(\theta)\big) = -l(\theta|y, x) + \Omega(\theta)$$

- The term $\Omega(\theta)$ captures our prior believes of what θ should be.

Probabilistic Intuition

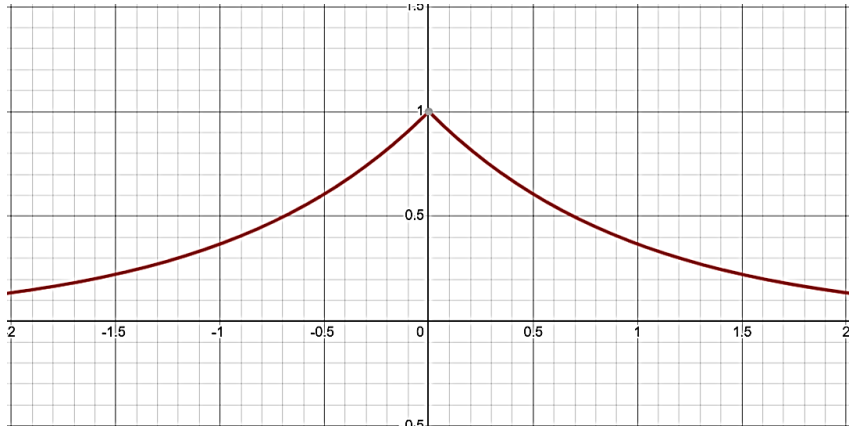
- If we assume a Gaussian prior center at 0, then

$$\Omega(\theta) = \lambda \sum_i \theta_i^2$$



- If we assume a Laplacian prior, the

$$\Omega(\theta) = \lambda \sum_i |\theta_i|$$



Types of Regularization

- **Parameter Regularization**
  - Parameter penalization
  - Transfer learning
- **Structure Regularization**
  - Parameter sharing
  - Batch normalization
  - Dropout
- **Data Regularization**
  - Data augmentation
  - Local flatness
  - Physics Informed

- For parameter penalization, think of what we discussed with $\Omega(\theta)$ with a squares or absolute value regularizer.
- Transfer learning captures some expectations on our parameters: the values can be shared across different tasks in the case of just using the feature extraction portion, and that the parameters are not far from each other across application in the case of fine-tuning.

Types of Regularization

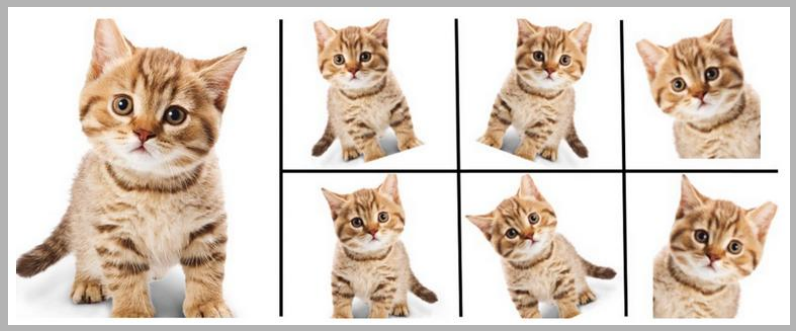
- **Parameter Regularization**
  - Parameter penalization
  - Transfer learning
- **Structure Regularization**
  - Parameter sharing
  - Batch normalization
  - Dropout
- **Data Regularization**
  - Data augmentation
  - Local flatness
  - Physics Informed

- In this case, $\Omega(\theta, S)$ where S is the net structure
- Parameter sharing enforces our expectation on the values of θ by imposing a structure on the net.
- **Batch normalization** enforces expectations on the distributions of θ by incorporating additional structure on the net.
- **Dropout** enforce some expectations on the structure of the net by adding and removing components of it.

Types of Regularization

- **Parameter Regularization**
  - Parameter penalization
  - Transfer learning
- **Structure Regularization**
  - Parameter sharing
  - Batch normalization
  - Dropout
- **Data Regularization**
  - Data augmentation
  - Local flatness
  - Physics Informed

- In this case, $\Omega(\theta, x)$
- **Data augmentation** enforces our belief that the model should provide the same output under small perturbations of the input (e.g., by adding noise, applying image transformation, adding synthetic context to real images, creating synthetic data).



Types of Regularization

- **Parameter Regularization**
  - Parameter penalization
  - Transfer learning
- **Structure Regularization**
  - Parameter sharing
  - Batch normalization
  - Dropout
- **Data Regularization**
  - Data augmentation
  - Local flatness
  - Physics Informed

- This is related to the concept of **local flatness** in which this invariance to small perturbations of the input will be enforces as a penalization rather than data generation.
- **Physics-Informed Neural Nets** (PINN) enforce our expectation that models that are predicting physical quantities should follow certain dynamic constraints, e.g., prediction of flow rates in a channel should satisfy Navier-Stokes equations.

Dropout

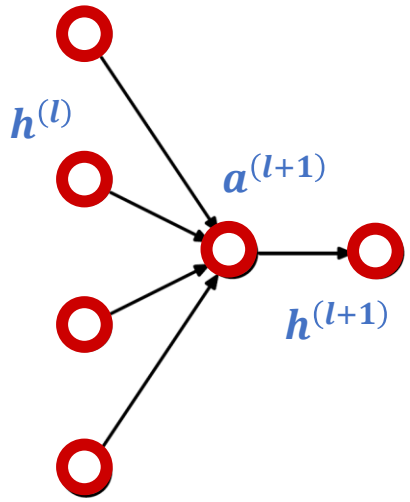
- It would be great to be able to run multiple neural nets (as an ensemble) to get more accurate prediction. However, due to computational cost that is not possible.

- How about running multiple models during training instead?

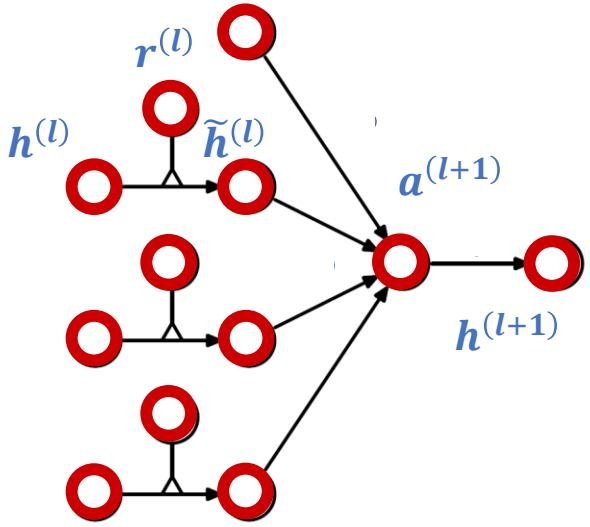
- **Dropout** achieves this randomly modifying the architecture of a neural net.

Dropout

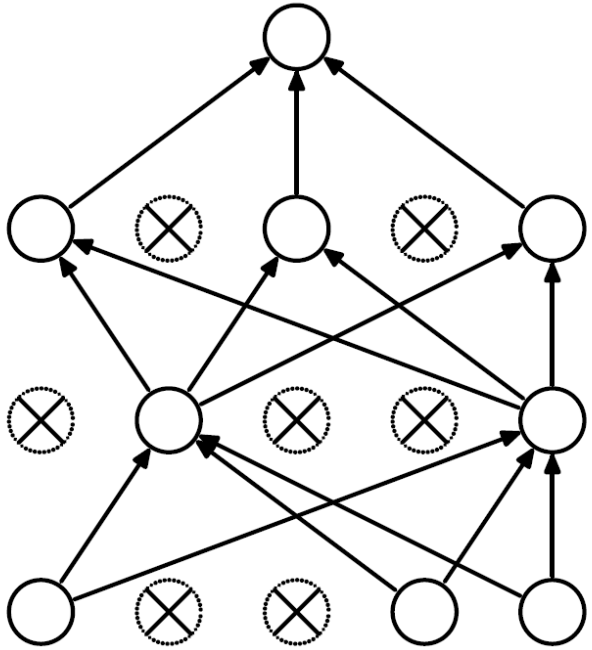
- Let's drop neurons at random during training.



Standard Network



Network with Dropout



- This is a type of regularization that reduces the over-reliance on specific features.